University of Tahri Mohamed Bechar
Faculty of Exact Sciences
Department of Computer Science

Operating System

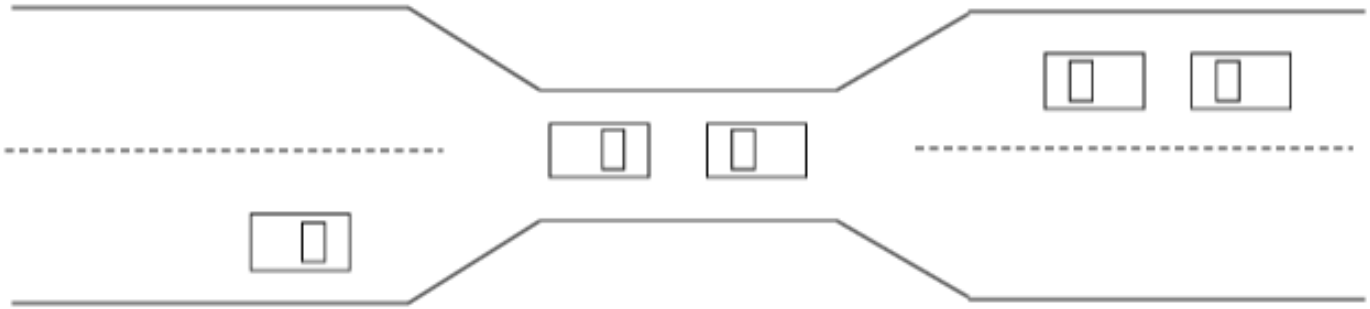
Deadlocks

Dr. Soumia BENKRAMA
benkrama.soumia@univ-bechar.dz

Introduction

• Generally speaking, deadlock, involves conflicting needs for resources by two or more request orders. A common example is a traffic deadlock.

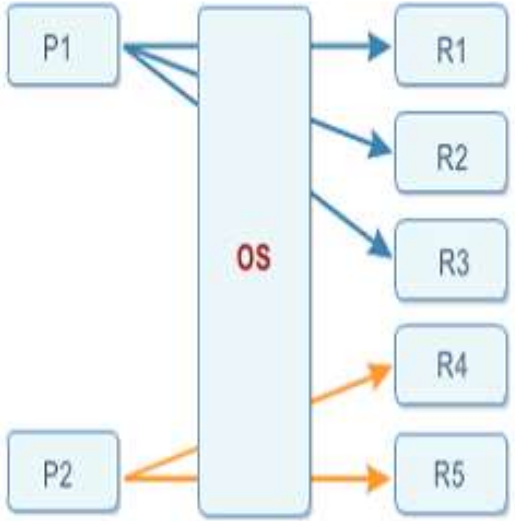
➢ If a deadlock occurs, it can be resolved if one car backs up (preempt resources and rolls back).
➢ Several cars may have to back up if a deadlock occurs.
➢ Starvation is possible.



Introduction

Deadlock in computer system

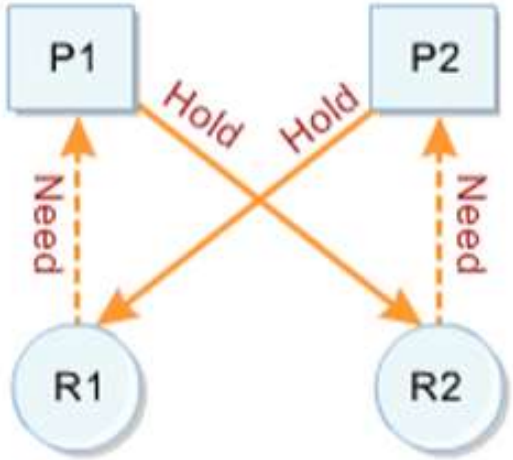
- A computer system consists of a **finite number of resources** to be distributed among a number of competing processes.
- An operating system is a **resource allocator** i.e., there are many resources that can be allocated to only one process at a time.
- Each process utilizes a resource as follows
  - **request**
  - **use**
  - **release**

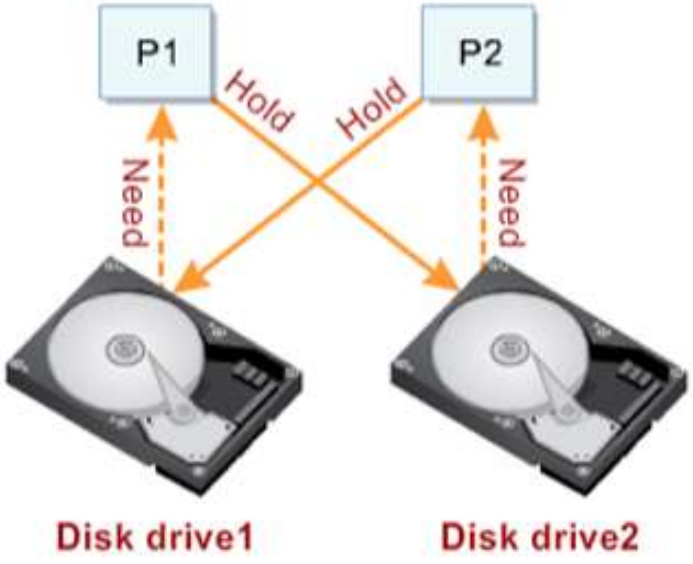


Introduction

Deadlock in computer system

- General example of deadlock in a computer system:





Introduction

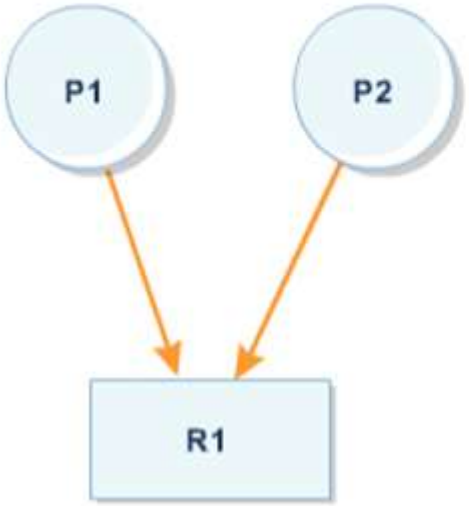
Types of resources in computer

- Resource types *R1*, *R2*, . . ., *Rm*
  CPU cycles, memory space, I/O devices

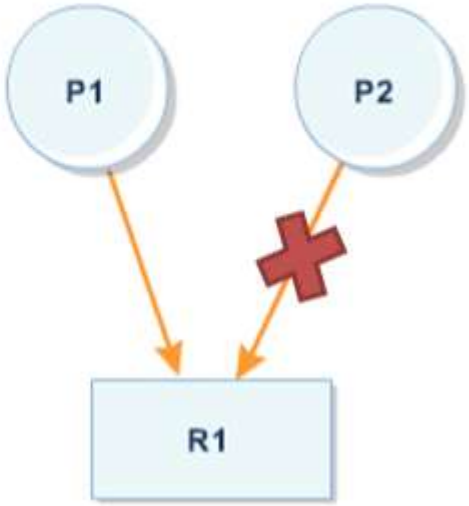
- Each resource type *Ri* has *Wi* instances.

Types of resources in computer

- **Sharable resources** can be used by more than one process at a time. A **consumable resource** can only be used by one process.



Sharable resources



Consumable resources

Types of resources in computer

- Resources can be pre-emptable or non pre-emptable.
  - Memory is an example of a **pre-emptable** resource, but

  - A printer is a **non-preemptable** one.

Types of resources in computer

- **Reusable**: used by one process at a time and then returned.
  - Processors, I/O channels, main and secondary memory, files, databases, and semaphores.
  - Deadlock may occur if each process holds one resource and requests another.

Types of resources in computer

- **Consumable**: created (produced) and destroyed (consumed) by a process.
  - Interrupts, signals, messages, data in I/O buffers.
  - Deadlock may occur if receive_message() is blocking.

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion.**
- **Hold and wait.**
- **No preemption.**
- **Circular wait.**

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

1. **Mutual exclusion:** only one process at a time can use a resource.

2. **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes.

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

3. No preemption: a resource can be released only willingly by the process holding it after that process has completed its task.

4. Circular wait: there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0

.

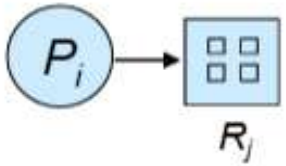
Deadlock Characterization

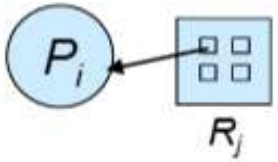
Resource Allocation Graph

- A set of vertices V and a set of edges E.
- V is partitioned into two types:
  - $P = \{P_1, P_2, \ldots, P_n\}$, the set consisting of all processes in the system.
  - $R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system.
- **Request edge** – directed edge $P_i \rightarrow R_j$
- **Assignment edge** – directed edge $R_j \rightarrow P_i$

Deadlock Characterization

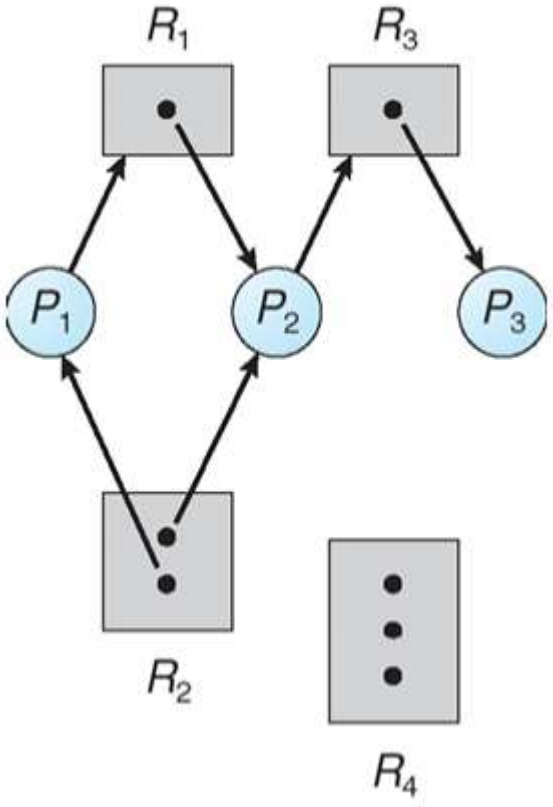
Resource Allocation Graph

- Process
- Resource Type with 4 instances
- P_i requests instance of R_j 

- P_i is holding an instance of R_j 

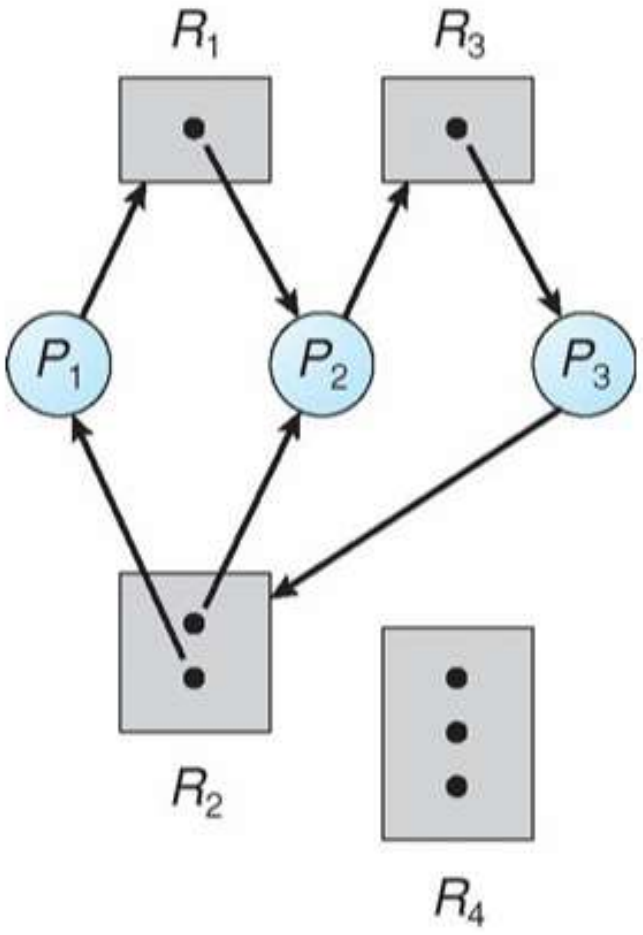
Deadlock Characterization

Example of a Resource Allocation Graph



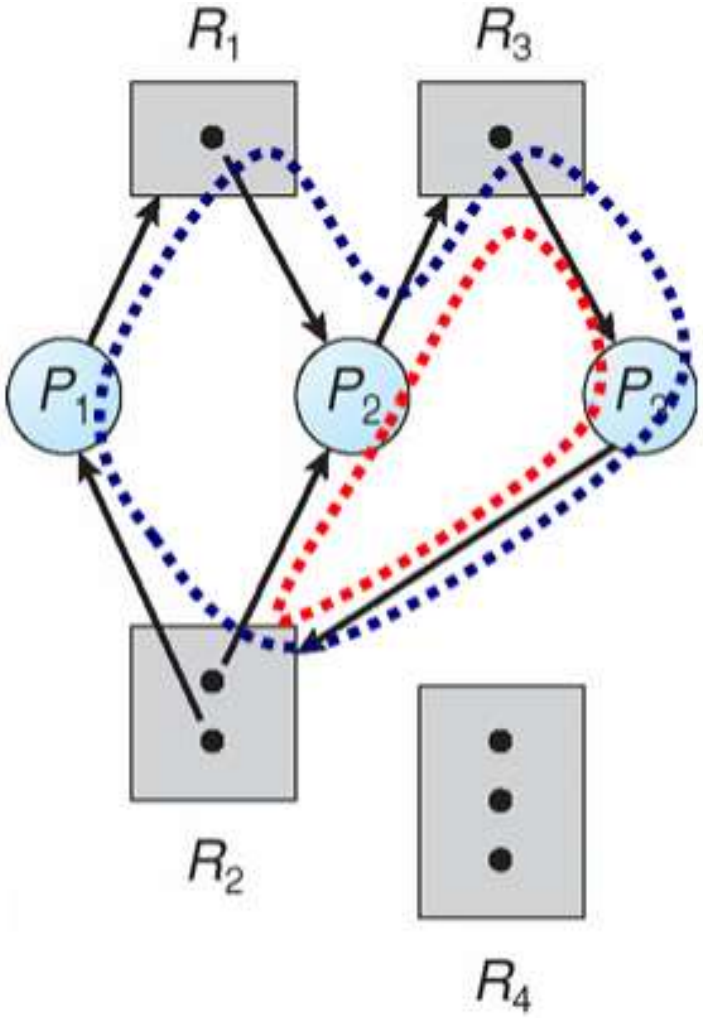
Deadlock Characterization

Example Resource Allocation Graph with A Deadlock



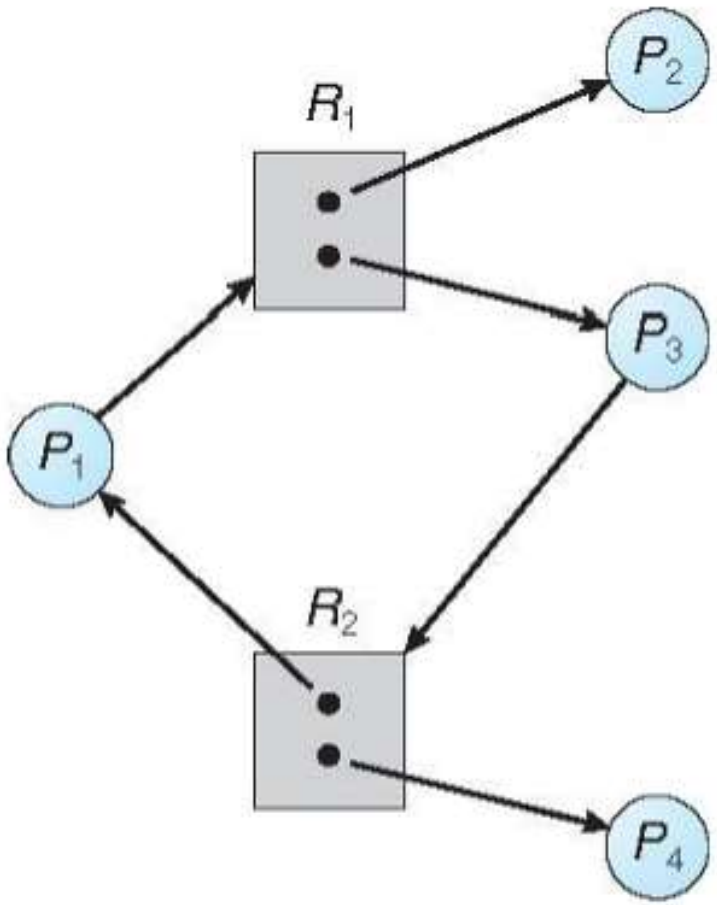
Deadlock Characterization

Resource Allocation Graph with A Deadlock



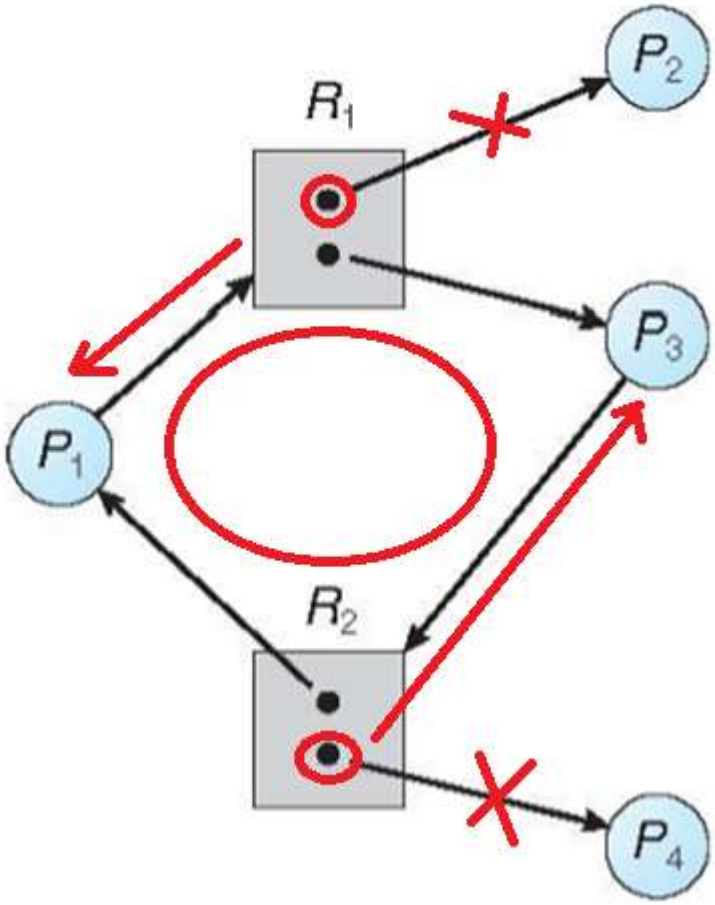
Deadlock Characterization

Graph with A Cycle but No Deadlock



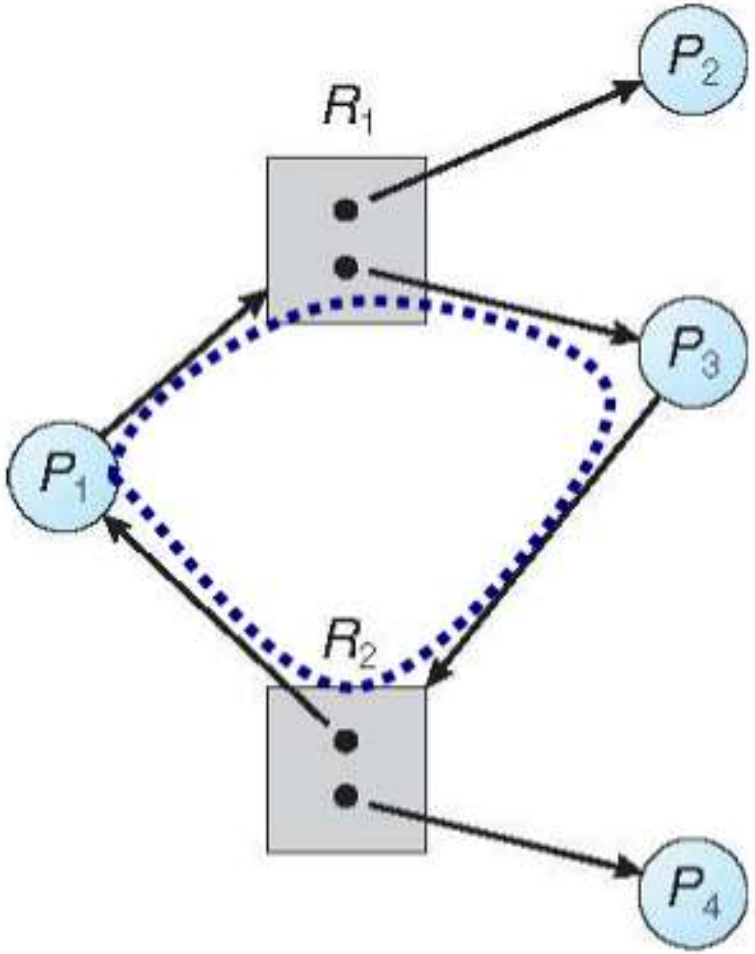
Deadlock Characterization

Graph with A Cycle but No Deadlock



Deadlock Characterization

Graph with A Cycle but No Deadlock



Deadlock Characterization

Basic Facts

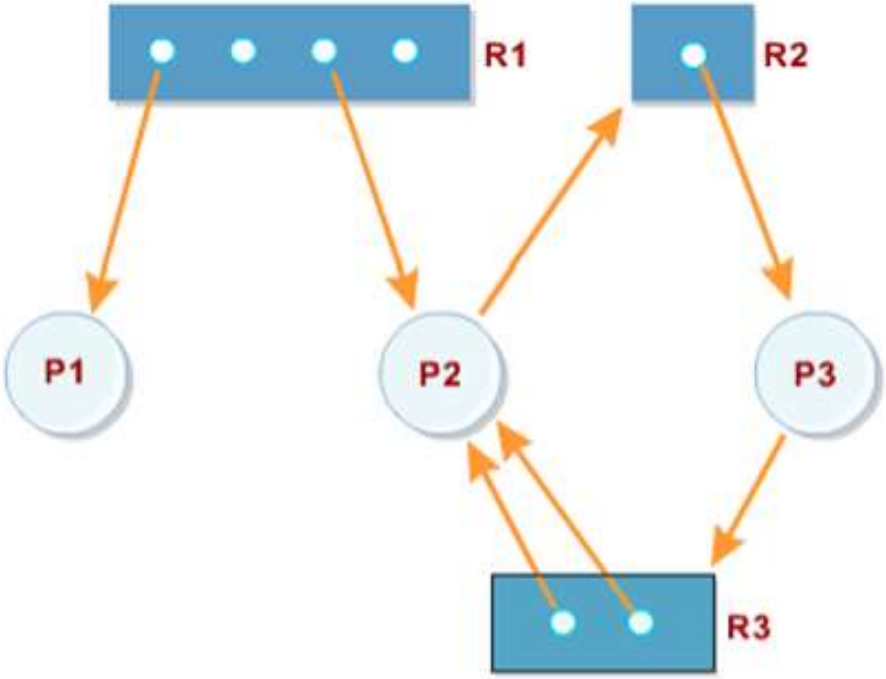
If graph contains **no cycles** ⇒ **no deadlock.**

If graph contains a **cycle** ⇒

- if only one instance per resource type, then deadlock.
- if several instances per resource type, possibility of deadlock.

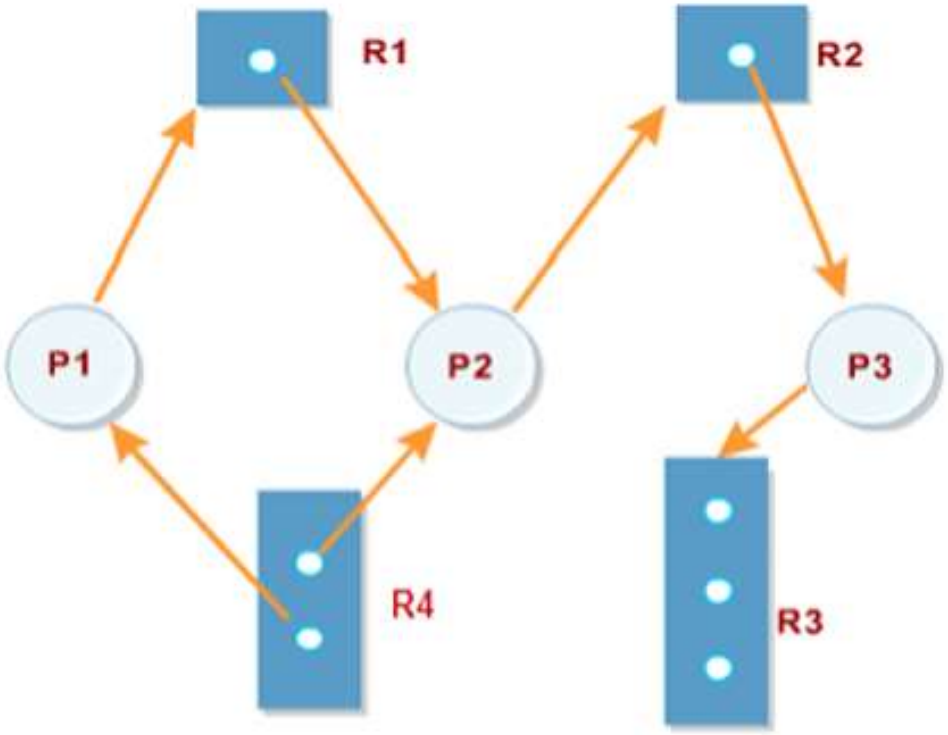
Deadlock Characterization

With a Deadlock or Without ??



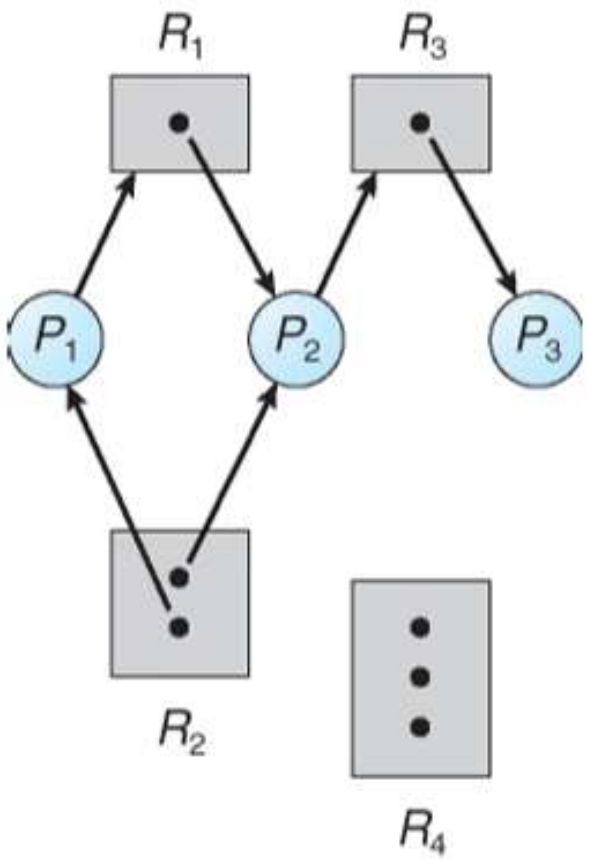
Deadlock Characterization

With a Deadlock or Without ??



Example Deadlocks

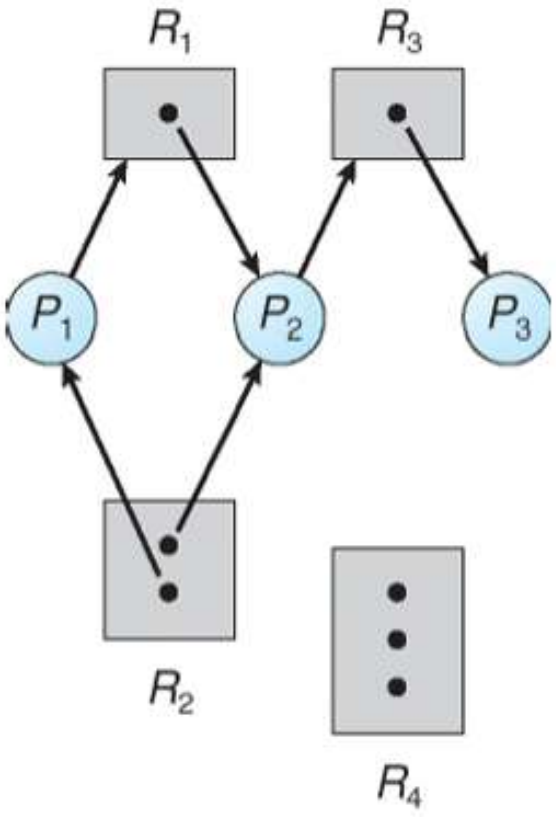
1. Describe the following Resource-Allocation Graph.
2. Is this graph contain a cycle ?
3. Is there a deadlock? Why?



Example Deadlocks

1. Describe the following Resource-Allocation Graph.

- There are 3 processes P_1, P_2, P_3
- There are 4 resources:
  - R_1(1 instance),
  - R_2(2 instances),
  - R_3(1 instance),
  - R_4(3 instances).
- P_1 holding 1 instance from R_2
- P_1 request 1 instance from R_1
- P_2 holding 1 instance from R_2
- P_2 holding 1 instance from R_1
- P_2 request 1 instance from R_3
- P_3 holding 1 instance from R_3



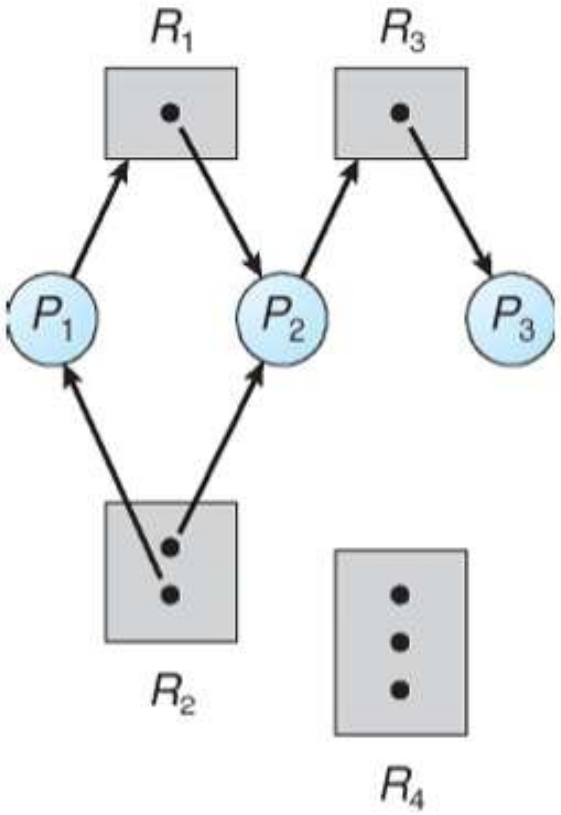
Example Deadlocks

2. Is this graph contain a cycle ?

- No cycle.

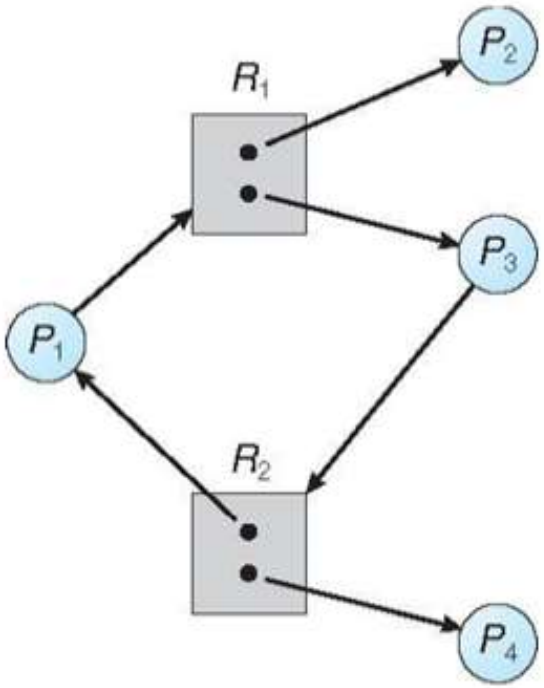
3. Is there a deadlock? Why?

- No deadlock. Because there is no cycle.



Example Deadlocks

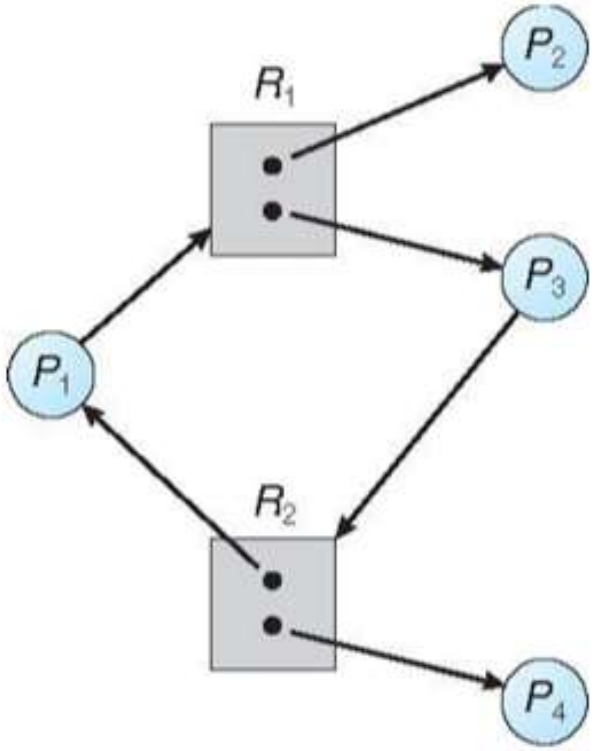
1. Describe the following Resource-Allocation Graph.
2. Is this graph contain a cycle ?
3. Is there a deadlock? Why?



Example Deadlocks

1. Describe the following Resource-Allocation Graph.

- There are 4 processes P_1, P_2, P_3, P_4
- There are 2 resources:
  - R_1 (2 instances),
  - R_2 (2 instances).
- P_1 holding 1 instance from R_2
- P_1 request 1 instance from R_1
- P_2 holding 1 instance from R_1
- P_3 holding 1 instance from R_1
- P_3 request 1 instance from R_2
- P_4 holding 1 instance from R_2



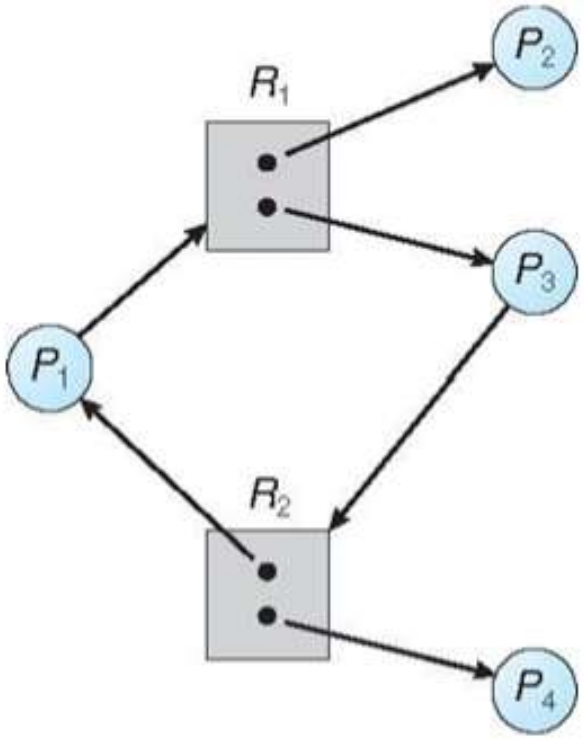
Example Deadlocks

2. Is this graph contain a cycle ?

- There is one cycle $< R_2, P_1, R_1, P_3, R_2 >$.

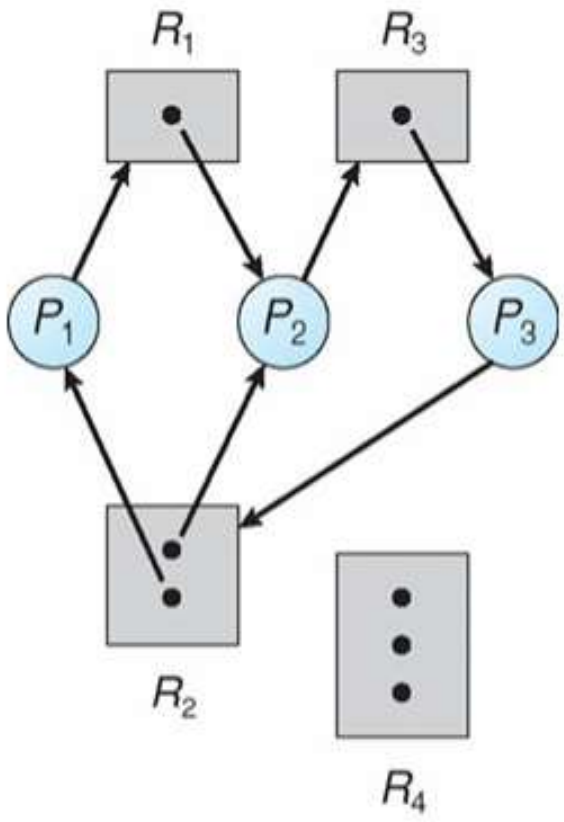
3. Is there a deadlock? Why?

- No deadlock.
- Because P_2 and P_4 will finish their jobs over time.
- Then, 1 instance from R_1 and 1 instance from R_2 will be free for P_1 and P_3.



Example Deadlocks

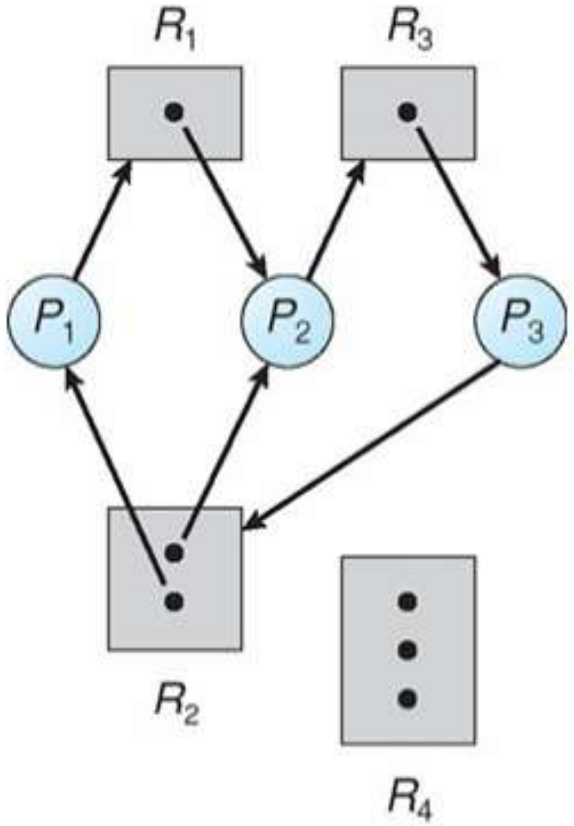
1. Describe the following Resource-Allocation Graph.
2. Is this graph contain a cycle ?
3. Is there a deadlock? Why?



Example Deadlocks

1. Describe the following Resource-Allocation Graph.

- There are 3 processes P_1, P_2, P_3
- There are 4 resources:
  - R_1(1 instance),
  - R_2(2 instances),
  - R_3(1 instance),
  - R_4(3 instances).
- P_1 holding 1 instance from R_2
- P_1 request 1 instance from R_1
- P_2 holding 1 instance from R_2
- P_2 holding 1 instance from R_1
- P_2 request 1 instance from R_3
- P_3 holding 1 instance from R_3
- P_3 request 1 instance from R_2



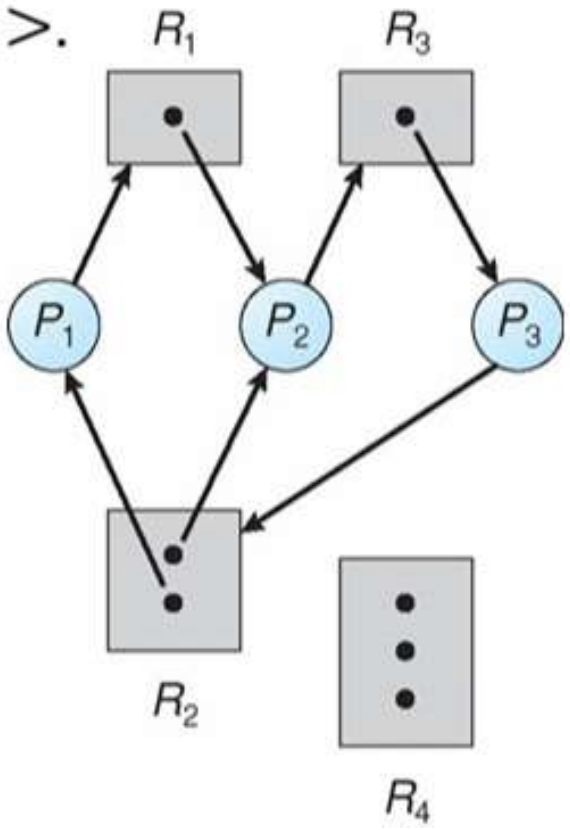
Example Deadlocks

2. Is this graph contain a cycle ?

- There is one cycle $< R_2, P_1, R_1, P_2, R_3, P_3, R_2 >$.

3. Is there a deadlock? Why?

- Yes there is a deadlock.
- Because P_1 is waiting for P_2 and,
- P_2 is waiting for P_3 and,
- P_3 is waiting for P_1 and P_2. Over time.



Methods for Handling Deadlocks

- Ensure that the system will **never** enter a deadlock state:
  - ➢ Deadlock prevention.
  - ➢ Deadlock avoidance.

- Allow the system to enter a deadlock state and then recover.

- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems.

Deadlock avoidance

Requires that the system has some additional a **priori** information available.

- Simplest and most useful model requires that each process declare the **maximum number** of resources of each type that it may need.
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.
- Resource-allocation **state** is defined by the number of available and allocated resources, and the maximum demands of the processes.

Deadlock avoidance

- When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state.
- System is in **safe state** if there exists a sequence $<P_1, P_2, \ldots, P_n>$ of ALL the processes in the systems such that for each P_i, the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j, with $j < i$.

Deadlock avoidance

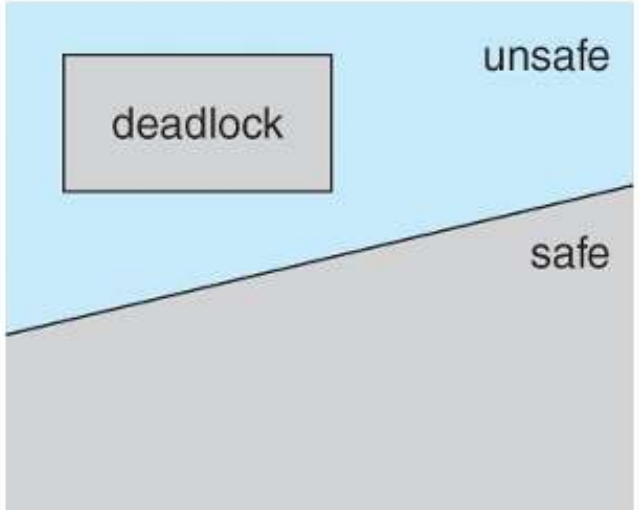
That is:

- If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished.
- When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate.
- When P_i terminates, P_{i+1} can obtain its needed resources, and so on.

Deadlock avoidance

Basic Facts

- If a system is in safe state ⇒ no deadlocks.
- If a system is in unsafe state ⇒ possibility of deadlock.
- Avoidance ⇒ ensure that a system will never enter an unsafe state.



Deadlock avoidance

Avoidance Algorithms:

- Single instance of a resource type ⇒
  - Use a resource-allocation graph.
- Multiple instances of a resource type ⇒
  - Use the banker's algorithm.

Deadlock avoidance

Resource-Allocation Graph:

- **Claim edge** $P_i \rightarrow R_j$ indicated that process P_i may request resource R_j; represented by a dashed line.

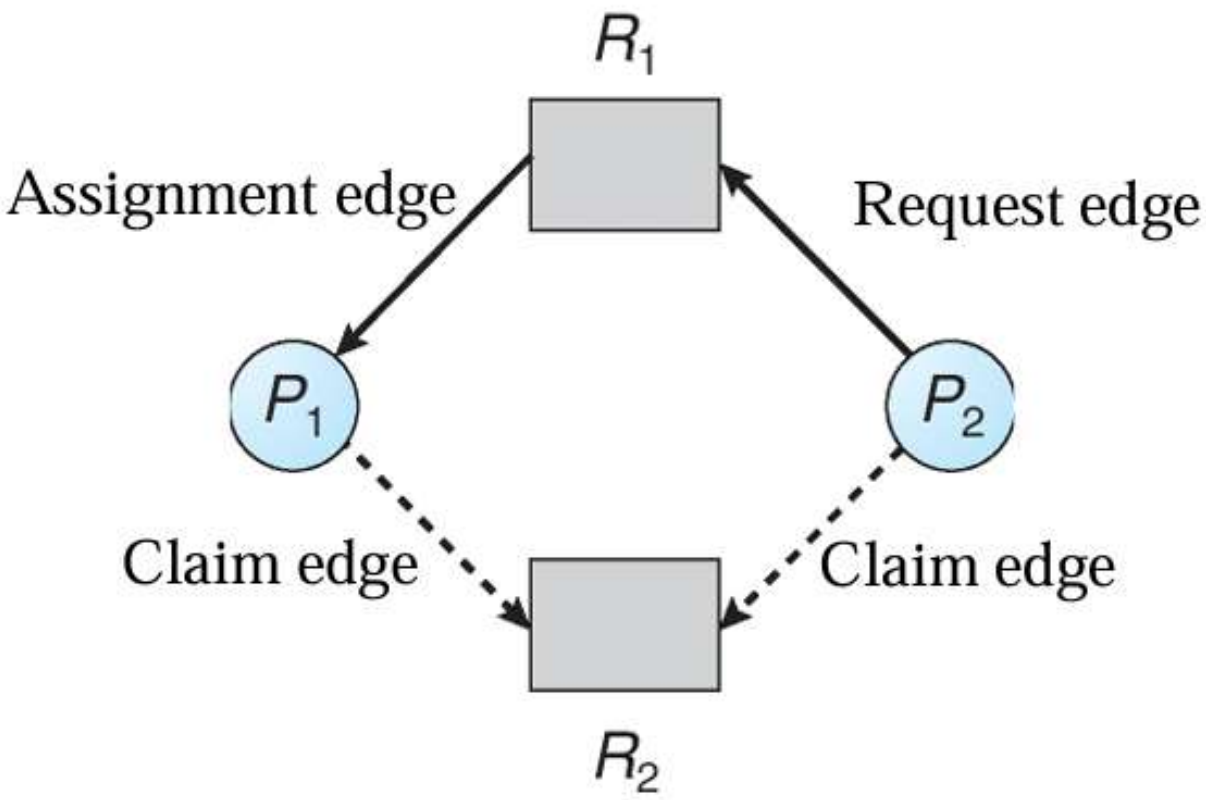
- Claim edge converts to request edge when a process requests a resource.

Deadlock avoidance

- Request edge converted to an assignment edge when the resource is allocated to the process.
- When a resource is released by a process, assignment edge reconverts to a claim edge.

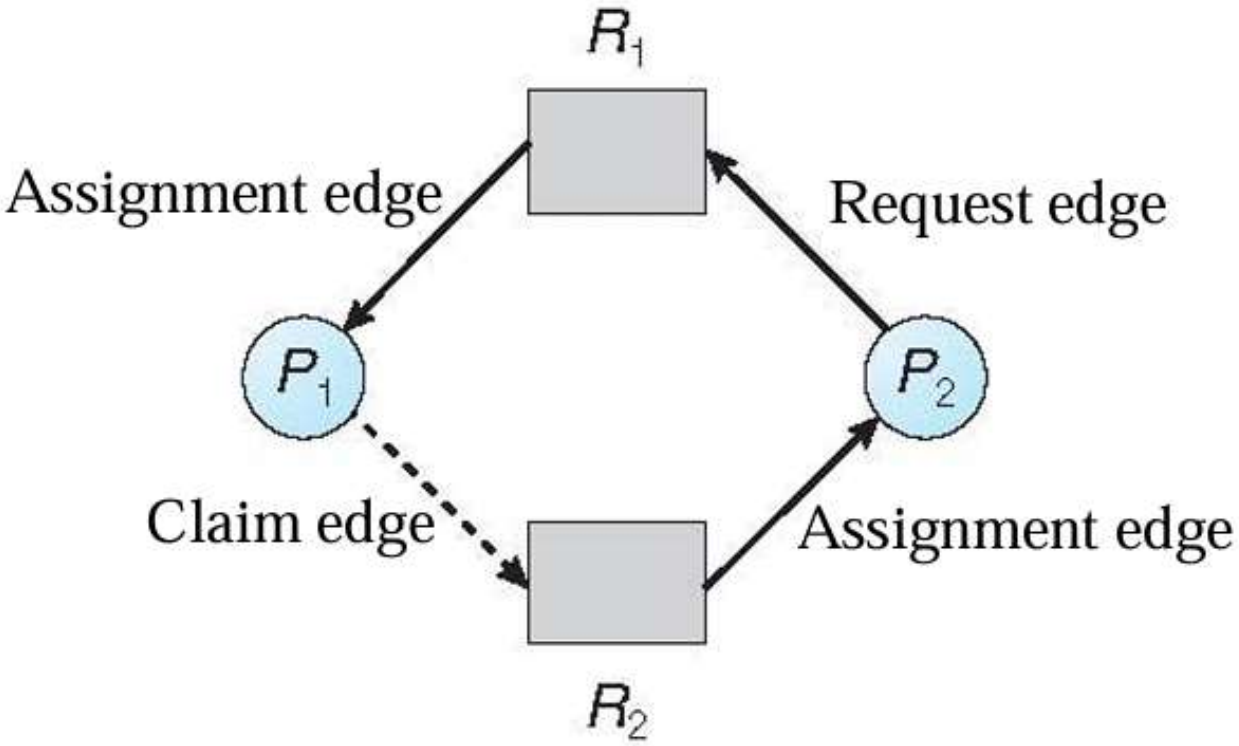
Deadlock avoidance

Resource-Allocation Graph:



Deadlock avoidance

Unsafe State In Resource-Allocation Graph:



A cycle, as mentioned, indicates that the system is in an unsafe state. If P_1 requests R_2, and P_2 holding R_2, then a deadlock will occur.

Deadlock avoidance

Resource-Allocation Graph Algorithm:

- Suppose that process P_i requests resource R_j
- The request can be granted only if converting the request edge to assignment edge does **not create a cycle** in the resource allocation graph.

Deadlock avoidance

Banker's Algorithm:

- Multiple instances of resources.
- Each process must a priori claim maximum use.
- When a process requests a resource it may have to wait.
- When a process gets all its resources it must return them in a finite amount of time.

Deadlock avoidance

Data Structures for the Banker's Algorithm:

Let n = number of processes, and m = number of resources types.

- **Available**: Vector of length m. If available $[j] = k$, there are k instances of resource type R_j available
- **Max**: $n \times m$ matrix. If $Max\,[i,j] = k$, then process P_i may request at most k instances of resource type R_j
- **Allocation**: $n \times m$ matrix. If Allocation$[i,j] = k$ then P_i is currently allocated k instances of R_j
- **Need**: $n \times m$ matrix. If $Need[i,j] = k$, then P_i may need k more instances of R_j to complete its task

$$Need\,[i,j] = Max[i,j] - Allocation\,[i,j]$$

Deadlock avoidance

Banker's Algorithm (Safety Algorithm):

1. Let ***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively. Initialize:

   Work = ***Available***

   Finish **[*i*]** = ***false*** **for** ***i*** **= 0, 1, …, *n*- 1**

2. Find an ***i*** such that both:

   (a) ***Finish*** **[*i*]** = ***false***

   (b) $Need_i \leq Work$

   If no such ***i*** exists, go to step 4

3. $Work = Work + Allocation_i$

   Finish**[*i*]** = ***true***

   go to step 2

4. If ***Finish*** **[*i*]** == ***true*** for all ***i***, then the system is in a safe state.

Deadlock avoidance

Banker's Algorithm (Resource-Request Algorithm):

Request$_i$ = request vector for process P_i. If ***Request***$_i$ [*j*] = *k* then process P_i wants *k* instances of resource type R_j

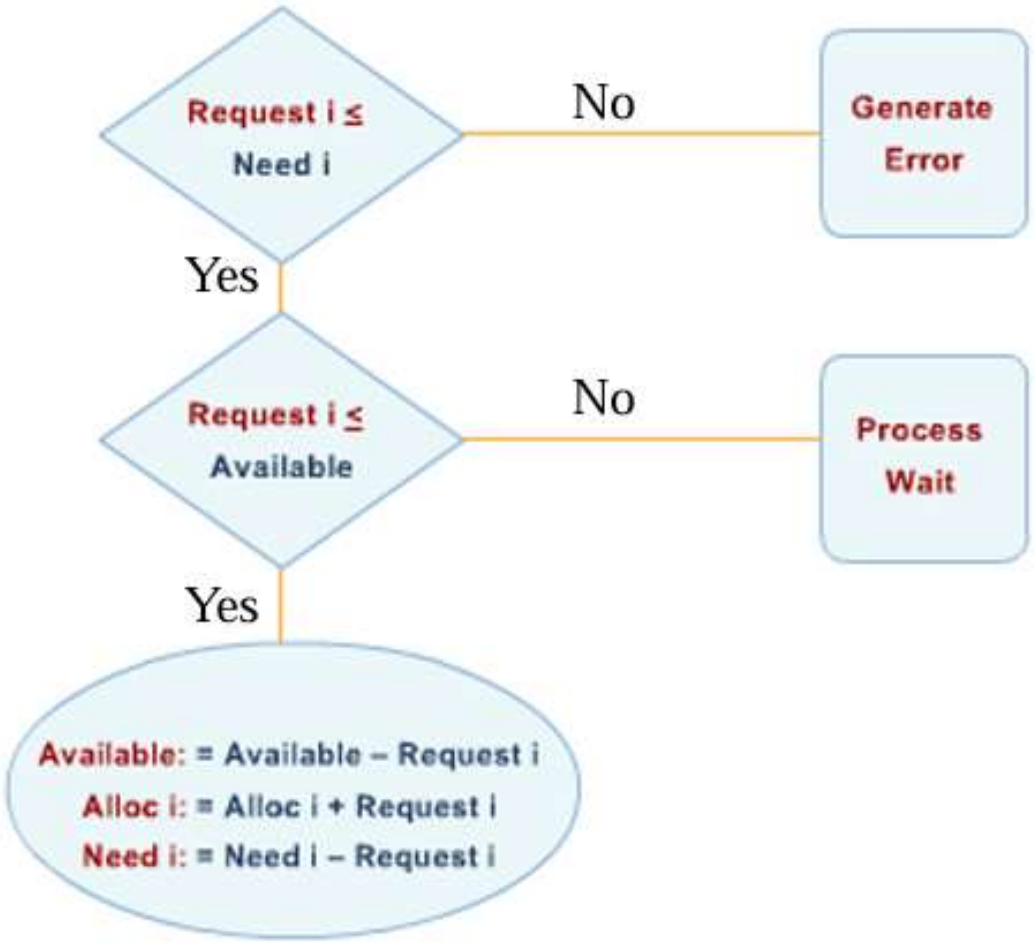
1. If ***Request***$_i \leq$ ***Need***$_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim.
2. If ***Request***$_i \leq$ ***Available***, go to step 3. Otherwise P_i must wait, since resources are not available.
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$\textit{Available} = \textit{Available} - \textit{Request}_i;$$
$$\textit{Allocation}_i = \textit{Allocation}_i + \textit{Request}_i;$$
$$\textit{Need}_i = \textit{Need}_i - \textit{Request}_i;$$

- If safe $\Rightarrow$ the resources are allocated to P_i.
- If unsafe $\Rightarrow$ P_i must wait, and the old resource-allocation state is restored.

Deadlock avoidance

Banker's Algorithm (Resource-Request Algorithm):



Deadlock avoidance

Banker's Algorithm (Example):

Consider the following snapshot of a system:

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

a. What is the content of the matrix ***Need***?

b. Is the system in a safe state? Why?

Deadlock avoidance

Banker's Algorithm (Example):

The content of the matrix ***Need*** is defined to be ***Max*** – ***Allocation***

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		
P_2	3 0 2	9 0 2		
P_3	2 1 1	2 2 2		
P_4	0 0 2	4 3 3		

Deadlock avoidance

Banker's Algorithm (Example):

The content of the matrix ***Need*** is defined to be ***Max*** – ***Allocation***

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		
P_3	2 1 1	2 2 2		
P_4	0 0 2	4 3 3		

Deadlock avoidance

Banker's Algorithm (Example):

The content of the matrix ***Need*** is defined to be ***Max*** – ***Allocation***

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

b. Is the system in a safe state? Why?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

b. Is the system in a safe state? Why?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

b. Is the system in a safe state? Why?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

b. Is the system in a safe state? Why?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	5 3 2	7 4 3
1 ~~P_1~~	~~2 0 0~~	~~3 2 2~~		~~1 2 2~~
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

b. Is the system in a safe state? Why?

		Allocation	*Max*	*Available*	*Need*
		A B C	*A B C*	*A B C*	*A B C*
	P_0	0 1 0	7 5 3	5 3 2	7 4 3
1	P_1	2 0 0	3 2 2		1 2 2
	P_2	3 0 2	9 0 2		6 0 0
	P_3	2 1 1	2 2 2		0 1 1
	P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

b. Is the system in a safe state? Why?

		Allocation	*Max*	*Available*	*Need*
		A B C	*A B C*	*A B C*	*A B C*
	P_0	0 1 0	7 5 3	5 3 2	7 4 3
1	~~P_1~~	~~2 0 0~~	~~3 2 2~~		~~1 2 2~~
	P_2	3 0 2	9 0 2		6 0 0
	P_3	2 1 1	2 2 2		0 1 1
	P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

b. Is the system in a safe state? Why?

		Allocation	*Max*	*Available*	*Need*
		A B C	*A B C*	*A B C*	*A B C*
	P_0	0 1 0	7 5 3	7 4 3	7 4 3
1	~~P_1~~	~~2 0 0~~	~~3 2 2~~		~~1 2 2~~
	P_2	3 0 2	9 0 2		6 0 0
2	~~P_3~~	~~2 1 1~~	~~2 2 2~~		~~0 1 1~~
	P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

b. Is the system in a safe state? Why?

		Allocation	*Max*	*Available*	*Need*
		A B C	*A B C*	*A B C*	*A B C*
3	P_0	0 1 0	7 5 3	7 5 3	7 4 3
1	P_1	2 0 0	3 2 2		1 2 2
	P_2	3 0 2	9 0 2		6 0 0
2	P_3	2 1 1	2 2 2		0 1 1
	P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

b. Is the system in a safe state? Why?

		Allocation	*Max*	*Available*	*Need*
		A B C	*A B C*	*A B C*	*A B C*
3	P_0	0 1 0	7 5 3	10 5 5	7 4 3
1	P_1	2 0 0	3 2 2		1 2 2
4	P_2	3 0 2	9 0 2		6 0 0
2	P_3	2 1 1	2 2 2		0 1 1
	P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

b. Is the system in a safe state? Why?

		Allocation	*Max*	*Available*	*Need*
		A B C	*A B C*	*A B C*	*A B C*
3	P_0	0 1 0	7 5 3	10 5 7	7 4 3
1	P_1	2 0 0	3 2 2		1 2 2
4	P_2	3 0 2	9 0 2		6 0 0
2	P_3	2 1 1	2 2 2		0 1 1
5	P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

b. Is the system in a safe state? Why?

		Allocation	*Max*	*Available*	*Need*
		A B C	*A B C*	*A B C*	*A B C*
3	P_0	0 1 0	7 5 3	10 5 7	7 4 3
1	P_1	2 0 0	3 2 2		1 2 2
4	P_2	3 0 2	9 0 2		6 0 0
2	P_3	2 1 1	2 2 2		0 1 1
5	P_4	0 0 2	4 3 3		4 3 1

The system is in a **safe state** since the sequence $< P_1, P_3, P_0, P_2, P_4 >$ **satisfies safety criteria**.

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	0 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	6 0 2	9 0 2		3 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	0 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	6 0 2	9 0 2		3 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	0 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	6 0 2	9 0 2		3 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	0 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	6 0 2	9 0 2		3 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	0 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	6 0 2	9 0 2		3 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	2 4 3	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	6 0 2	9 0 2		3 0 0
1 P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

		Allocation	*Max*	*Available*	*Need*
		A B C	*A B C*	*A B C*	*A B C*
	P_0	0 1 0	7 5 3	4 4 3	7 4 3
2	~~P_1~~	~~2 0 0~~	~~3 2 2~~		~~1 2 2~~
	P_2	6 0 2	9 0 2		3 0 0
1	~~P_3~~	~~2 1 1~~	~~2 2 2~~		~~0 1 1~~
	P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

		Allocation	*Max*	*Available*	*Need*
		A B C	*A B C*	*A B C*	*A B C*
	P_0	0 1 0	7 5 3	4 4 3	7 4 3
2	~~P_1~~	~~2 0 0~~	~~3 2 2~~		~~1 2 2~~
	P_2	6 0 2	9 0 2		3 0 0
1	~~P_3~~	~~2 1 1~~	~~2 2 2~~		~~0 1 1~~
	P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

		Allocation	*Max*	*Available*	*Need*
		A B C	*A B C*	*A B C*	*A B C*
	P_0	0 1 0	7 5 3	10 4 5	7 4 3
2	P_1	2 0 0	3 2 2		1 2 2
3	P_2	6 0 2	9 0 2		3 0 0
1	P_3	2 1 1	2 2 2		0 1 1
	P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

		Allocation	*Max*	*Available*	*Need*
		A B C	*A B C*	*A B C*	*A B C*
4	P_0	0 1 0	7 5 3	10 5 5	7 4 3
2	P_1	2 0 0	3 2 2		1 2 2
3	P_2	6 0 2	9 0 2		3 0 0
1	P_3	2 1 1	2 2 2		0 1 1
	P_4	0 0 2	4 3 3		4 3 1

Deadlock avoidance

Banker's Algorithm (Example):

c. If a request from process P_2 arrives for (3,0,0), can the request be granted immediately?

		Allocation	*Max*	*Available*	*Need*
		A B C	*A B C*	*A B C*	*A B C*
4	P_0	0 1 0	7 5 3	10 5 7	7 4 3
2	P_1	2 0 0	3 2 2		1 2 2
3	P_2	6 0 2	9 0 2		3 0 0
1	P_3	2 1 1	2 2 2		0 1 1
5	P_4	0 0 2	4 3 3		4 3 1

Yes you can granted this request immediately, because the system will be in a **safe state** since the sequence < P_3, P_1, P_2, P_0, P_4> **satisfies safety criteria**.

Deadlock avoidance

Banker's Algorithm (Example):

d. If a request from process P_0 arrives for (3,2,2), can the request be granted immediately?

	Allocation	*Max*	*Available*	*Need*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Deadlock detection

If a system does not employ either a deadlock-prevention or a deadlock avoidance algorithm, then a deadlock situation may occur. In this environment, the system may provide:

- An algorithm that examines the state of the system to determine whether a deadlock has occurred.
- An algorithm to recover from the deadlock.

Deadlock detection

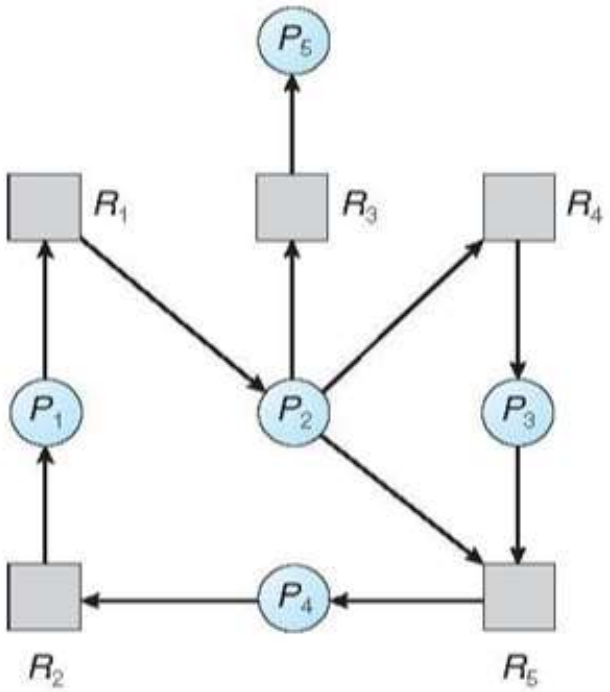
Single Instance of Each Resource Type:

- Maintain **wait-for** graph:
  - Nodes are processes.
  - $P_i \rightarrow P_j$ if P_i is waiting for P_j.
- Periodically invoke an algorithm that searches for a **cycle** in the graph. If there is a cycle, there exists a deadlock.
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph.

Deadlock detection

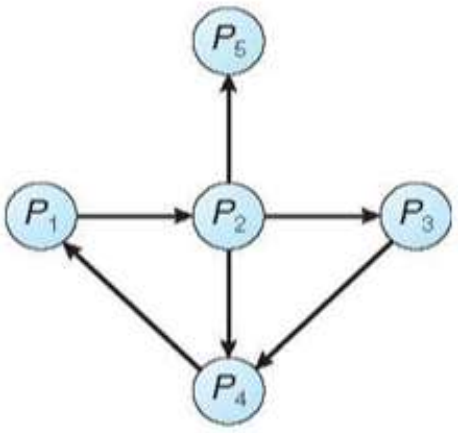
Single Instance of Each Resource Type:

- Resource-Allocation Graph and Wait-for Graph



(a)

Resource-Allocation Graph



(b)

Corresponding wait-for graph

Deadlock detection

Several Instance of a Resource Type:

- **Available**: A vector of length m indicates the number of available resources of each type.
- **Allocation**: An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
- **Request**: An $n \times m$ matrix indicates the current request of each process. If $Request\ [i][j] = k$, then process P_i is requesting k more instances of resource type R_j.

Deadlock detection

Detection Algorithm: (1/2)

1. Let ***Work*** and ***Finish*** be vectors of length m and n, respectively
   Initialize:
   (a) $Work = Available$
   (b) For $i = 1,2, \ldots, n$, if $Allocation_i \neq 0$, then
   $Finish[i] = false$; otherwise, $Finish[i] = true$

2. Find an index i such that both:
   (a) $Finish[i] == false$
   (b) $Request_i \leq Work$

   If no such i exists, go to step 4

Deadlock detection

Detection Algorithm: (2/2)

3. $Work = Work + Allocation_i$
   $Finish[i] = true$
   go to step 2

4. If $Finish[i] == false$, for some i, $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if $Finish[i] == false$, then P_i is deadlocked.

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state.

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*
	A B C	*A B C*	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2		
P_2	3 0 3	0 0 0		
P_3	2 1 1	1 0 0		
P_4	0 0 2	0 0 2		

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*
	A B C	*A B C*	*A B C*	*A B C*	
P_0	0 1 0	0 0 0	0 0 0	0 0 0	
P_1	2 0 0	2 0 2			
P_2	3 0 3	0 0 0			
P_3	2 1 1	1 0 0			
P_4	0 0 2	0 0 2			

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*
	A B C	*A B C*	*A B C*	*A B C*	
P_0	0 1 0	0 0 0	0 0 0	0 0 0	false
P_1	2 0 0	2 0 2			false
P_2	3 0 3	0 0 0			false
P_3	2 1 1	1 0 0			false
P_4	0 0 2	0 0 2			false

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*
	A B C	*A B C*	*A B C*	*A B C*	
P_0	0 1 0	0 0 0	0 0 0	0 0 0	false
P_1	2 0 0	2 0 2			false
P_2	3 0 3	0 0 0			false
P_3	2 1 1	1 0 0			false
P_4	0 0 2	0 0 2			false

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*
	A B C	*A B C*	*A B C*	*A B C*	
P_0	0 1 0	0 0 0	0 0 0	0 0 0	false
P_1	2 0 0	2 0 2			false
P_2	3 0 3	0 0 0			false
P_3	2 1 1	1 0 0			false
P_4	0 0 2	0 0 2			false

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*
	A B C	*A B C*	*A B C*	*A B C*	
P_0	0 1 0	0 0 0	0 0 0	0 0 0	false
P_1	2 0 0	2 0 2			false
P_2	3 0 3	0 0 0			false
P_3	2 1 1	1 0 0			false
P_4	0 0 2	0 0 2			false

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 1 0	0 0 0	0 0 0	0 1 0	true	1
P_1	2 0 0	2 0 2			false	
P_2	3 0 3	0 0 0			false	
P_3	2 1 1	1 0 0			false	
P_4	0 0 2	0 0 2			false	

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	0 1 0	true	1
P_1	2 0 0	2 0 2			false	
P_2	3 0 3	0 0 0			false	
P_3	2 1 1	1 0 0			false	
P_4	0 0 2	0 0 2			false	

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	0 1 0	true	1
P_1	2 0 0	2 0 2			false	
P_2	3 0 3	0 0 0			false	
P_3	2 1 1	1 0 0			false	
P_4	0 0 2	0 0 2			false	

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	0 1 0	true	1
P_1	2 0 0	2 0 2			false	
P_2	3 0 3	0 0 0			false	
P_3	2 1 1	1 0 0			false	
P_4	0 0 2	0 0 2			false	

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	3 1 3	true	1
P_1	2 0 0	2 0 2			false	
P_2	3 0 3	0 0 0			true	2
P_3	2 1 1	1 0 0			false	
P_4	0 0 2	0 0 2			false	

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	3 1 3	true	1
P_1	2 0 0	2 0 2			false	
P_2	0 0 0	0 0 0			true	2
P_3	2 1 1	1 0 0			false	
P_4	0 0 2	0 0 2			false	

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	3 1 3	true	1
P_1	2 0 0	2 0 2			false	
P_2	0 0 0	0 0 0			true	2
P_3	2 1 1	1 0 0			false	
P_4	0 0 2	0 0 2			false	

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	3 1 3	true	1
P_1	2 0 0	2 0 2			false	
P_2	0 0 0	0 0 0			true	2
P_3	2 1 1	1 0 0			false	
P_4	0 0 2	0 0 2			false	

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	5 2 4	true	1
P_1	2 0 0	2 0 2			false	
P_2	0 0 0	0 0 0			true	2
P_3	2 1 1	1 0 0			true	3
P_4	0 0 2	0 0 2			false	

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	5 2 4	true	1
P_1	2 0 0	2 0 2			false	
P_2	0 0 0	0 0 0			true	2
P_3	0 0 0	0 0 0			true	3
P_4	0 0 2	0 0 2			false	

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	5 2 4	true	1
P_1	2 0 0	2 0 2			false	
P_2	0 0 0	0 0 0			true	2
P_3	0 0 0	0 0 0			true	3
P_4	0 0 2	0 0 2			false	

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	5 2 6	true	1
P_1	2 0 0	2 0 2			false	
P_2	0 0 0	0 0 0			true	2
P_3	0 0 0	0 0 0			true	3
P_4	0 0 2	0 0 2			true	4

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	5 2 6	true	1
P_1	2 0 0	2 0 2			false	
P_2	0 0 0	0 0 0			true	2
P_3	0 0 0	0 0 0			true	3
P_4	0 0 0	0 0 0			true	4

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	5 2 6	true	1
P_1	2 0 0	2 0 2			false	
P_2	0 0 0	0 0 0			true	2
P_3	0 0 0	0 0 0			true	3
P_4	0 0 0	0 0 0			true	4

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	7 2 6	true	1
P_1	2 0 0	2 0 2			true	5
P_2	0 0 0	0 0 0			true	2
P_3	0 0 0	0 0 0			true	3
P_4	0 0 0	0 0 0			true	4

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	7 2 6	true	1
P_1	0 0 0	0 0 0			true	5
P_2	0 0 0	0 0 0			true	2
P_3	0 0 0	0 0 0			true	3
P_4	0 0 0	0 0 0			true	4

Deadlock detection

Detection Algorithm (Example1):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*	*Work*	*Finish*	
	A B C	*A B C*	*A B C*	*A B C*		
P_0	0 0 0	0 0 0	0 0 0	7 2 6	true	1
P_1	0 0 0	0 0 0			true	5
P_2	0 0 0	0 0 0			true	2
P_3	0 0 0	0 0 0			true	3
P_4	0 0 0	0 0 0			true	4

We claim that the system is **not** in a **deadlocked** state. Indeed, if we execute our algorithm, we will find that the sequence < P_0, P_2, P_3, P_4, P_1> results in *Finish*[*i*] == *true* for all *i*.

Deadlock detection

Detection Algorithm (Example 2):

a. Is the system is in deadlock state? Why?

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	**0 0 1**	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Recovery from Deadlock

Recovery from Deadlock:

- Process Termination.
- Resource Preemption.

Recovery from Deadlock

Process Termination:

1. Abort all deadlocked processes.
2. Abort one process at a time until the deadlock cycle is eliminated.

Recovery from Deadlock

Process Termination:

3. In which order should we choose to abort?
   - Priority of the process.
   - How long process has computed, and how much longer to completion.
   - Resources the process has used.
   - Resources process needs to complete.
   - How many processes will need to be terminated.
   - Is process interactive or batch?

Recovery from Deadlock

Resource Preemption:

- If preemption is required to deal with deadlocks, then three issues need to be addressed:

  1. **Selecting a victim** – Which resources and which processes are to be preempted? (minimize cost).

  2. **Rollback** – return to some safe state, restart process for that state.

  3. **Starvation** – same process may always be picked as victim, include number of rollback in cost factor.